# جزوه دوره مهندسی داده و مهندسی هوش مصنوعی

**Data Engineering & AI Engineering**

یادداشت‌های آموزشی، مفاهیم فنی و تمرین‌های دوره




| |
|---|
| جلسه هشتم:SQL (قسمت دوم) |
| مدرس :کوثر اصغری |
| تاریخ :سه‌شنبه ۵ آبان ۱۴۰۵ |

تاریخ به روز رسانی : 20261028

# درباره این جزوه

این جزوه با هدف ثبت، سازمان‌دهی و بازنویسی آموزشی مطالب دوره رایگان «مهندسی داده و مهندسی هوش مصنوعی» تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثال‌های مطرح‌شده در کلاس و پرسش‌وپاسخ‌های آموزشی تدوین می‌شود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبت‌های مدرس تغییر کند.

هدف این جزوه صرفاً خلاصه‌کردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آماده‌شدن برای فعالیت‌های عملی به آن مراجعه کند.

# رویکرد آموزشی و شیوه تدوین

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده می‌شوند. سپس مثال‌های مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی می‌شوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- **مطالب اصلی جلسه :**بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثال‌های اصلی.
- **تکمیل فنی :**توضیحات دقیق‌تر، اصلاح برداشت‌های احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.
- **مثال اضافی :**مثال‌ها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شده‌اند.
- **نکات کاربردی :**ملاحظات اجرایی، محدودیت‌ها و تجربه‌های مرتبط با استفاده عملی از مفاهیم.
- **جمع‌بندی و سوال‌های مرور :**مرور مهم‌ترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شده‌اند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسان‌تر باشد.

# محدوده و توسعه جزوه

این مجموعه به‌صورت تدریجی و هم‌زمان با انتشار جلسات جدید تکمیل می‌شود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش می‌دهد.

نسخه‌های به‌روزشده جزوه‌ها، منابع تکمیلی و یادداشت‌های هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود.

**مخزن جزوه‌ها:**

https://github.com/LaneZero/data-ai-engineering-course

**وب‌سایت اصلی دوره:**

https://mostafaghadimi.github.io/data-course/

**یادداشت درباره اعتبار محتوا**

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثال‌های افزوده‌شده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه می‌شود هنگام اجرای پروژه‌های واقعی، اطلاعات حساس یا تصمیم‌گیری‌های فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، **کمک به یادگیری عمیق‌تر، مرور منظم مفاهیم و اشتراک دانش با علاقه‌مندان حوزه داده و هوش مصنوعی** است.

# فهرست مطالب


۱. هدف جلسه و نقشه راه ..... ٤

۲. مرور تمرین: تعداد سفارش به تفکیک وضعیت ..... ٤

۳. تفکر مجموعه‌ای در SQL ..... ٥

۴. Window Function: محاسبه بدون از بین بردن ردیف‌ها ..... ٦

۴.۱ شماره سفارش، سفارش قبلی و جمع تجمعی ..... ٦

۴.۲ مجموع و درصد هر سفارش از خرید مشتری ..... ٧

۴.۳ پیدا کردن اولین سفارش هر مشتری ..... ٨

۵. شش خطای رایج و خطرناک در SQL ..... ٨

۵.۱ خطای اول: Fan-out پس از JOIN ..... ٩

۵.۲ خطای دوم: تبدیل ناخواسته LEFT JOIN به فیلتر محدودکننده ..... ١٠

۵.۳ خطای سوم: NULL و منطق سه‌ارزشی ..... ١١

۵.۴ خطای چهارم: NOT IN و حضور NULL در زیرپرس‌وجو ..... ١١

۵.۵ خطای پنجم: بازه زمانی و دقت timestamp ..... ١٢

۵.۶ خطای ششم: UPDATE بدون شرط و بررسی اثر تغییر ..... ١٣

۶. تله‌های عددی و تقسیم بر صفر ..... ١٤

۷. ساخت کوئری پارامتری و جلوگیری از خطاهای امنیتی ..... ١٤

۸. الگوهای تحلیلی که مدرس برای تمرین معرفی کرد ..... ١٥

۸.۱ رتبه‌بندی: ROW_NUMBER, RANK و DENSE_RANK ..... ١٥

۸.۲ حذف تکراری‌ها ..... ١٦

۸.۳ رشد ماهانه و جمع تجمعی ..... ١٧

۸.۴ قیف مشتری، Point-in-Time و الگوهای پیشرفته ..... ١٨

۹. پیوند SQL با مهندسی داده: Grain, Fact و Dimension ..... ١٨

۱۰. چک‌لیست پیش از تأیید Query ..... ١٩

۱۱. تجربه مدرس و توصیه‌های پایانی ..... ١٩

**۱۲. جمع‌بندی** ..... ۱۹
**۱۳. سوال‌های مرور** ..... ۲۰
**۱۴. پاسخ‌نامه کوتاه** ..... ۲۰
**۱۵. واژه‌نامه** ..... ۲۱

## ۱. هدف جلسه و نقشه راه

این جلسه ادامه آموزش SQL است. مدرس ابتدا یک تمرین تجمیعی از جلسه پیش را حل می‌کند، سپس مفهوم «تفکر مجموعه‌ای» را با اجرای چندباره Query در حلقه مقایسه می‌کند. بخش اصلی به Window Functions اختصاص دارد و در ادامه، خطاهایی بررسی می‌شوند که گاهی بدون هیچ خطای نحوی، نتیجه گزارش را نادرست می‌کنند: Fan-out، شرط نامناسب روی LEFT JOIN، رفتار NULL، بازه زمانی، به‌روزرسانی بی‌شرط، تقسیم عددی و ساخت Query در برنامه. در پایان نیز الگوهای تحلیلی و ارتباط SQL با Fact, Dimension و Grain مرور می‌شود.

بعد از مطالعه باید بتوانید Window Function را از GROUP BY تشخیص دهید، برای «اولین سفارش هر مشتری» Query بنویسید، خطای تکثیر ردیف پس از JOIN را پیدا کنید، NULL را درست فیلتر کنید، بازه زمانی دقیق بسازید و قبل از اجرای UPDATE اثر آن را بررسی کنید.

- **تکمیل فنی:** نمونه‌های اجرایی این جزوه با نحو PostgreSQL نوشته شده‌اند؛ دیتاست آموزشی مدرس در فایل همراه این جلسه موجود نیست. نام ستون‌ها از کلاس استخراج شده‌اند و مثال‌های افزوده ممکن است با جدول‌های واقعی نیاز به تطبیق داشته باشند.

## ۲. مرور تمرین: تعداد سفارش به تفکیک وضعیت

سوال مدرس: «تعداد سفارش‌ها در هر وضعیت طی سال ۲۰۲۶ را حساب کن؛ فقط وضعیت‌هایی که بیش از ۲۰۰۰ سفارش دارند نمایش بده و از بیشترین تعداد به کمترین مرتب کن.» هر ردیف جدول `orders` یک سفارش است؛ پس برای شمارش سفارش‌ها، لازم نیست جدول دیگری JOIN شود.

```sql
SELECT
    status,
    COUNT(*) AS order_count
FROM
    orders
WHERE
    order_at >= TIMESTAMP '2026-01-01 00:00:00'
    AND order_at < TIMESTAMP '2027-01-01 00:00:00'
GROUP BY
    status
HAVING
    COUNT(*) > 2000
```

```
ORDER BY
    order_count DESC;
```

`WHERE` ردیف‌های خارج از سال را حذف می‌کند؛ `GROUP BY` سفارش‌ها را بر اساس وضعیت دسته‌بندی می‌کند؛ `HAVING` روی تعداد هر گروه شرط می‌گذارد و `ORDER BY` نتیجه را مرتب می‌کند. در کلاس نشان داده شد که حذف `HAVING`، وضعیت‌هایی با شمارش کمتر را هم برمی‌گرداند.

- **تکمیل فنی:** بازه درست برای کل سال، «شروع شامل، پایان خارج» است. `order_at < '2027-01-01'` تمام ساعت‌های روز آخر ۲۰۲۶ را پوشش می‌دهد. در متن گفتار گاهی «کوچک‌تر مساوی» شنیده می‌شود؛ در شرط بالا عمداً از `<` استفاده شده است.

## ۳. تفکر مجموعه‌ای در SQL

مدرس دو روش را مقایسه کرد: اجرای Query جدا برای تک‌تک مشتریان در حلقه برنامه‌نویسی و اجرای یک Query که مجموع خرید همه مشتریان را یکجا حساب می‌کند. رویکرد دوم معمولاً رفت‌وبرگشت‌های شبکه‌ای کمتری دارد و به بهینه‌ساز پایگاه داده فرصت می‌دهد عملیات را یکپارچه اجرا کند.

```
SELECT
    customer_id,
    SUM(order_amount) AS total_amount
FROM
    orders
GROUP BY
    customer_id;
```

در روش حلقه‌ای، برای هر شناسه مشتری یک درخواست مستقل به پایگاه داده ارسال می‌شود. این الگو می‌تواند به مشکل N+1 Query منتهی شود: تعداد زیادی درخواست کوچک، به جای یک درخواست مجموعه‌ای. اصل کلاس این است که قبل از نوشتن حلقه روی داده‌ها، ببینیم آیا SQL می‌تواند کل مسئله را یکجا حل کند.

- **تکمیل فنی:** یک Query لزوماً فقط «یک بار دیسک را می‌خواند» یا همیشه سریع‌تر نیست؛ موتور ممکن است چند Scan یا Index Lookup انجام دهد. مزیت رویکرد مجموعه‌ای به طرح اجرا، ایندکس‌ها، حجم داده و هزینه رفت‌وبرگشت هم وابسته است.

## ۴. Window Function: محاسبه بدون از بین بردن ردیف‌ها

با `GROUP BY` معمولاً چند سفارش هر مشتری به یک ردیف جمع‌بندی تبدیل می‌شوند. اما گاهی می‌خواهیم هر سفارش را نگه داریم و کنار آن، شماره سفارش، مبلغ سفارش قبلی، مجموع خرید همان مشتری یا سهم هر سفارش از کل خرید مشتری را نیز داشته باشیم. اینجا Window Function کاربرد دارد.

ساختار کلی:

```
FUNCTION_NAME(...) OVER(PARTITION BY customer_id
ORDER BY
    order_at,
    order_id)
```

`PARTITION BY` ردیف‌ها را به گروه‌های مستقل تقسیم می‌کند؛ `ORDER BY` ترتیب محاسبه را در هر گروه مشخص می‌کند. `OVER` تعیین می‌کند تابع روی کدام پنجره اجرا شود. این گروه‌بندی، برخلاف `GROUP BY`، لزوماً تعداد ردیف‌های خروجی را کم نمی‌کند.

### ۴.۱ شماره سفارش، سفارش قبلی و جمع تجمعی

مدرس سه کار را کنار هم نمایش داد: شماره‌گذاری سفارش‌های یک مشتری با `ROW_NUMBER`، خواندن مبلغ قبلی با `LAG` و حساب‌کردن مجموع خرید تا هر سفارش با `SUM OVER`.

```
SELECT
    customer_id,
    order_id,
    order_at,
    order_amount,
    ROW_NUMBER() OVER(PARTITION BY customer_id
ORDER BY
    order_at,
    order_id) AS order_number, LAG(order_amount) OVER(PARTITI
ON BY customer_id
ORDER BY
    order_at,
    order_id) AS previous_amount, SUM(order_amount) OVER(PART
ITION BY customer_id
ORDER BY
    order_at,
    order_id ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW) AS running_total
FROM
    orders
```

```
WHERE
    customer_id = 1
ORDER BY
    order_at,
    order_id;
```

در اولین سفارش، `previous_amount` معمولاً `NULL` است؛ چون سفارش قبلی وجود ندارد. ستون `running_total` با هر سفارش به مجموع قبلی اضافه می‌شود. در کلاس تأکید شد که فهم `PARTITION BY` و تفاوت آن با گروه‌بندی معمولی، به تمرین نیاز دارد.

- **تکمیل فنی:** اگر چند سفارش زمان یکسان داشته باشند، `ORDER BY order_at` به‌تنهایی ترتیب قطعی ایجاد نمی‌کند. افزودن `order_id` به عنوان معیار دوم، ترتیب محاسبه را پایدارتر می‌کند. همچنین فریم صریح `ROWS` از ابهام رفتار پیش‌فرض بعضی موتورهای پایگاه داده در مواجهه با زمان‌های برابر جلوگیری می‌کند.

## ۴.۲ مجموع و درصد هر سفارش از خرید مشتری

در مثال بعد، جمع کل خرید هر مشتری در کنار مبلغ تک‌تک سفارش‌های همان مشتری نمایش داده شد. این بار پنجره فقط `PARTITION BY` دارد تا تمام سفارش‌های مشتری در یک گروه جمع شوند.

```
SELECT
    customer_id,
    order_id,
    order_amount,
    SUM(order_amount) OVER(PARTITION BY customer_id) AS custo
mer_total,
    ROUND(100.0 * order_amount / NULLIF(SUM(order_amount) OVE
R(PARTITION BY customer_id), 0), 2) AS percent_of_customer
FROM
    orders;
```

در یک مشتری با چند سفارش، `customer_total` برای همه سفارش‌ها تکرار می‌شود، اما `order_amount` هر سطر مستقل است. درصد سهم هر سطر، نسبت مبلغ همان سفارش به مجموع کل خرید همان مشتری است.

- **تکمیل فنی:** در برخی SQL Dialectها نمی‌توان یک Window Function را مستقیماً داخل Window Function دیگر تو در تو نوشت. مثال بالا دو بار یک Window Aggregate مستقل استفاده می‌کند، نه Window تو در تو. `NULLIF` مانع تقسیم بر صفر می‌شود و آن را به نتیجه `NULL` تبدیل می‌کند.

### ۴.۳ پیدا کردن اولین سفارش هر مشتری

مدرس نشان داد که نام مستعار `rn` در همان سطح Query معمولاً در `WHERE` قابل استفاده نیست: محاسبه Window Function در مرحله‌ای پس از فیلتر `WHERE` قرار می‌گیرد. راه‌حل، ساخت نتیجه میانی با CTE و سپس فیلتر کردن آن است.

```
WITH numbered_orders AS(
    SELECT
        customer_id,
        order_id,
        order_at,
        ROW_NUMBER() OVER(PARTITION BY customer_id
    ORDER BY
        order_at,
        order_id) AS rn
    FROM
        orders
)
SELECT
    customer_id,
    order_id,
    order_at
FROM
    numbered_orders
WHERE
    rn = 1;
```

`COUNT(*)` اگر فقط تعداد مشتریانی را بخواهیم که حداقل یک سفارش ثبت کرده‌اند، می‌توانیم از نتیجه میانی بگیریم. مدرس در دیتاست کلاس، شمارش ۴۲۵۰ مشتری سفارش‌دهنده را نمایش داد؛ این عدد به همان دیتاست وابسته است و عدد عمومی نیست.

> **مثال اضافی:** برای شمارش مشتریان دارای سفارش، Query ساده‌تر `SELECT COUNT(DISTINCT customer_id) FROM orders WHERE customer_id IS NOT NULL;` نیز ممکن است کافی باشد. CTE زمانی ارزش بیشتری دارد که خودِ اولین سفارش یا اطلاعات مربوط به آن را بخواهیم.

## ۵. شش خطای رایج و خطرناک در SQL

یکی از پیام‌های محوری این جلسه این است که Query ممکن است بدون Error اجرا شود، اما عددی بدهد که از نظر کسب‌وکار اشتباه است. مدرس شش دسته خطا را با مثال فروشگاهی بررسی کرد.

### ۵.۱ خطای اول: Fan-out پس از JOIN

هر ردیف `orders` یک سفارش است، اما جدول `order_items` می‌تواند برای هر سفارش چند ردیف داشته باشد. اگر قبل از جمع مبلغ سفارش‌ها، این دو جدول را وصل کنیم، مبلغ هر سفارش به تعداد قلم‌های آن تکرار می‌شود. JOIN دیگر لزوماً یک ردیف به ازای هر سفارش ندارد.

```
-- Potentially wrong: each order may repeat for each item.
SELECT
    SUM(o.order_amount) AS total_amount
FROM
    orders AS o
JOIN order_items AS oi
    ON oi.order_id = o.order_id
WHERE
    o.status = 'paid';
```

در کلاس مدرس عدد تجمیع بعد از JOIN را با جمع مستقیم سفارش‌های پرداخت‌شده مقایسه کرد و اختلاف قابل‌توجهی دید. برای تشخیص علت، تعداد ردیف‌ها و تعداد شناسه‌های متمایز سفارش پس از JOIN را مقایسه کرد.

```
SELECT
    COUNT(*) AS joined_rows,
    COUNT(DISTINCT o.order_id) AS unique_orders
FROM
    orders AS o
JOIN order_items AS oi
    ON oi.order_id = o.order_id;
```

- **تکمیل فنی:** علاوه بر تکرار شدن سفارش‌ها، `INNER JOIN` ممکن است سفارش‌های فاقد قلم مرتبط را حذف کند. بنابراین تعداد ردیف‌های خروجی می‌تواند بیشتر شود و تعداد سفارش‌های یکتای باقی‌مانده هم کمتر شود. یک CTE که قبل از JOIN جمع می‌زند، اگر بعداً به رابطه یک‌به‌چند JOIN شود و جمع دوباره محاسبه شود، به‌تنهایی مانع Fan-out نیست؛ باید Grain نتیجه نهایی حفظ شود.

**مثال اضافی:** اگر فقط وجود حداقل یک قلم برای سفارش مهم است، `EXISTS` از JOIN و تجمیع مجدد روشن‌تر است:

```
SELECT
    SUM(o.order_amount) AS total_amount
FROM
    orders AS o
```

```
WHERE
    o.status = 'paid'
    AND EXISTS (
    SELECT
        1
    FROM
        order_items AS oi
    WHERE
        oi.order_id = o.order_id
);
```

## ۵.۲ خطای دوم: تبدیل ناخواسته LEFT JOIN به فیلتر محدودکننده

سوال کلاس: «تعداد سفارش‌های پرداخت‌شده هر مشتری را، حتی برای مشتری بدون سفارش پرداخت‌شده، نمایش بده.» شرط وضعیت اگر در `WHERE` باشد، ردیف‌های NULL سمت راست را حذف می‌کند و مشتریان بدون سفارش پرداخت‌شده از پاسخ ناپدید می‌شوند.

```
SELECT
    c.customer_id,
    COUNT(o.order_id) AS paid_orders
FROM
    customers AS c
LEFT JOIN orders AS o
    ON o.customer_id = c.customer_id
    AND o.status = 'paid'
GROUP BY
    c.customer_id;
```

با گذاشتن شرط در `ON`، همه مشتریان سمت چپ باقی می‌مانند و برای مشتری بدون تطابق، شمارش `o.order_id` صفر می‌شود. مدرس در داده کلاس تفاوت ۴۲۵۰ ردیف با ۵۰۰۰ مشتری را برای نمایش اثر این جابه‌جایی نشان داد.

- **تکمیل فنی:** `COUNT(*)` پس از `LEFT JOIN` برای مشتری بدون سفارش نیز ردیف حاصل از JOIN را یک عدد حساب می‌کند. برای شمارش سفارش‌ها از `COUNT(o.order_id)` یا شمارش شرطی مناسب استفاده کنید. جابه‌جایی شرط از `WHERE` به `ON` باید با توجه به معنای گزارش انجام شود، نه به‌عنوان قانون همیشگی.

## ۵.۳ خطای سوم: NULL و منطق سه‌ارزشی

در کلاس، مدرس فیلتر «کشور برابر ایران نیست» را بررسی کرد. شرط زیر فقط ردیف‌هایی را نگه می‌دارد که مقایسه آن‌ها واقعاً TRUE باشد. ردیف‌های دارای `country IS NULL` نه مساوی ایران‌اند و نه نامساوی آن در منطق سه‌ارزشی SQL؛ نتیجه مقایسه UNKNOWN است.

```
SELECT
    COUNT(*) AS not_iran_known
FROM
    customers
WHERE
    country <> 'Iran';
```

اگر منظور گزارش، «تمام ردیف‌هایی است که صریحاً ایران نیستند یا کشور نامشخص دارند»، شرط باید این تصمیم را واضح کند:

```
SELECT
    COUNT(*) AS not_iran_or_missing
FROM
    customers
WHERE
    country <> 'Iran'
    OR country IS NULL;
```

مدرس برای تشریح این دام، تعداد ردیف‌های دارای مقادیر NULL در `country` و `city` را جداگانه شمرد و به تفاوت `COUNT(*)` با `COUNT(city)` اشاره کرد.

- **تکمیل فنی:** از نظر معنایی نمی‌توان با اطمینان گفت مشتری دارای کشور نامعلوم «ایرانی نیست». اضافه‌کردن `OR country IS NULL` انتخاب تحلیلی مشخصی است، نه تصحیح خودکار داده مجهول. در PostgreSQL، `country IS DISTINCT FROM 'Iran'` راهی فشرده برای مقایسه NULL-safe است.

## ۵.۴ خطای چهارم: NOT IN و حضور NULL در زیرپرس‌وجو

مدرس مثال «مشتریانی که شهرشان در فهرست شهرهای مشتریان ترکیه نیست» را مطرح کرد. اگر زیرپرس‌وجوی `NOT IN` حتی یک `NULL` داشته باشد، نتیجه مقایسه برای بسیاری از ردیف‌ها UNKNOWN می‌شود و ممکن است خروجی غیرمنتظره، حتی صفر، به دست بیاید.

```
-
- This may return unexpected results when the subquery has NU
LL.
SELECT
```

```
    COUNT(*)
FROM
    customers
WHERE
    city NOT IN (
    SELECT
        city
    FROM
        customers
    WHERE
        country = 'Turkey'
);
```

راهکار آموزشی جلسه، استفاده از `NOT EXISTS` با شرط ارتباطی بین دو جدول است:

```
SELECT
    COUNT(*)
FROM
    customers AS c
WHERE
    NOT EXISTS (
    SELECT
        1
    FROM
        customers AS t
    WHERE
        t.country = 'Turkey'
        AND t.city = c.city
);
```

- **تکمیل فنی:** در Query بالا، برای مشتری‌ای که `c.city IS NULL` است نیز تطابق برابری پیدا نمی‌شود و او در نتیجه باقی می‌ماند. اگر «شهر نامعلوم» نباید شمرده شود، شرط `c.city IS NOT NULL` را صریحاً اضافه کنید. این تصمیم باید با تعریف سوال کسب‌وکار هماهنگ باشد.

## ۵.۵ خطای پنجم: بازه زمانی و دقت timestamp

مدرس با سفارش‌های سپتامبر نشان داد که شرط `BETWEEN '2026-09-01' AND '2026-09-30'` در ستون Timestamp ممکن است فقط تا ابتدای روز ۳۰ سپتامبر را پوشش دهد، نه تمام روز را. حتی نوشتن `23:59:59` نیز رکوردهای دارای بخش کسری ثانیه را جا می‌اندازد.

```
SELECT
    SUM(order_amount) AS september_total
FROM
```

```
    orders
WHERE
    order_at >= TIMESTAMP '2026-09-01 00:00:00'
    AND order_at < TIMESTAMP '2026-10-01 00:00:00';
```

این الگوی نیمه‌باز `[start, end)` هم خواناست و هم با دقت‌های مختلف Timestamp سازگارتر است.

- **تکمیل فنی:** اگر ستون زمان از نوع `timestamptz` باشد یا داده از چند منطقه زمانی بیاید، مرز ماه را باید با Time Zone مورد نظر گزارش تعریف کرد. مثال بالا فرض می‌کند تعبیر زمان و محدوده گزارش یکسان است. عدد اختلاف گزارش‌شده در کلاس وابسته به دیتاست همان جلسه است.

## ۵.۶ خطای ششم: UPDATE بدون شرط و بررسی اثر تغییر

مدرس هشدار داد `UPDATE orders SET status = 'cancelled';` همه ردیف‌های جدول را تغییر می‌دهد. روش مناسب‌تر، اول مشخص‌کردن دامنه تغییر، سپس اجرای آن در Transaction و بررسی نتیجه پیش از نهایی‌سازی است.

```
BEGIN;
SELECT
    COUNT(*) AS candidate_rows
FROM
    orders
WHERE
    status = 'pending'
    AND order_at < TIMESTAMP '2025-02-01 00:00:00';
UPDATE orders
SET
    status = 'cancelled'
WHERE
    status = 'pending'
    AND order_at < TIMESTAMP '2025-02-01 00:00:00'
RETURNING
    order_id,
    status;
-- Inspect returned rows before choosing COMMIT or ROLLBACK.
ROLLBACK;
```

مدرس در داده کلاس تعداد ۴۷ سفارش کاندید را برای یک شرط زمانی نشان داد؛ این عدد در پایگاه داده‌های دیگر قابل تعمیم نیست.

- **تکمیل فنی:** `ROLLBACK` صرفاً تغییرات تراکنش باز را برمی‌گرداند؛ به معنی «اگر یکی از شرط‌های WHERE برقرار نبود، خودکار برگردان» نیست. اگر Query بدون Error اجرا شود و شرط تعداد ردیف را اشتباه نوشته باشیم، پایگاه داده الزاماً خطای منطقی را تشخیص نمی‌دهد. برای محیط عملیاتی، مجوز محدود، Backup، تراکنش و کنترل شمارش ضروری‌اند.

## ۶. تله‌های عددی و تقسیم بر صفر

مدرس نشان داد که نحوه محاسبه نسبت در SQL اهمیت دارد؛ در برخی Dialectها تقسیم دو Integer نتیجه صحیح می‌دهد و بخش اعشاری حذف می‌شود. برای نمایش نسبت عددی، یکی از عملوندها را به نوع اعشاری تبدیل کنید.

```
SELECT
    5 / 3 AS integer_division,
    5.0 / 3 AS decimal_division,
    5.0 / NULLIF(0, 0) AS safe_zero_division;
```

`NULLIF(x, 0)` وقتی مخرج صفر باشد NULL می‌دهد. در مثال‌های تحلیل خرید باید مراقب باشیم که `AVG`، `SUM`، تعداد سفارش و تعداد اقلام هرکدام Grain متفاوتی دارند؛ «میانگین تعداد قلم در هر سفارش» با «میانگین تعداد قلم در هر ردیف قلم» یکسان نیست.

- **تکمیل فنی:** رفتار دقیق Integer Division به موتور SQL و نوع عملوندها بستگی دارد. همچنین `NULL` به‌جای Error تصمیمی آگاهانه است؛ در برخی گزارش‌ها باید شمارش موارد تقسیم بر صفر را جداگانه پایش کرد، نه اینکه خطای داده بی‌صدا پنهان شود.

## ۷. ساخت کوئری پارامتری و جلوگیری از خطاهای امنیتی

مدرس درباره اجرای SQL از برنامه و ابزارهای خودکار صحبت کرد و تأکید داشت فیلترها باید در Query سمت پایگاه داده اعمال شوند؛ خواندن کل جدول و سپس فیلترکردن در برنامه می‌تواند هزینه شبکه و حافظه را افزایش دهد.

- **تکمیل فنی:** پارامتری‌کردن Query علاوه بر خوانایی، برای جلوگیری از SQL Injection ضروری است. جایگزین‌کردن مستقیم متن ورودی کاربر داخل SQL با String Concatenation امن نیست؛ باید از Parameter Binding در Driver یا ORM استفاده شود. پارامتری‌کردن به‌تنهایی تضمین نمی‌کند Query حتماً از Index استفاده کند.

**مثال اضافی:** نمونه Python با پارامتر و اتصال از پیش تنظیم‌شده:

```
# Bind parameters through the database driver; never concatenate user input.
email = user_supplied_email
cursor.execute(
    "SELECT customer_id, full_name FROM customers WHERE email = %s",
    (email,),
)
rows = cursor.fetchall()
```

## ۸. الگوهای تحلیلی که مدرس برای تمرین معرفی کرد

مدرس تأکید کرد این بخش قرار نیست در یک جلسه به تسلط برسد. هدف، شناخت خانواده سوال‌ها و استفاده بعدی از تمرین‌هاست.

### ۸.۱ رتبه‌بندی: ROW_NUMBER, RANK و DENSE_RANK

این سه تابع برای رتبه‌دادن استفاده می‌شوند، اما رفتارشان در مواجهه با امتیاز برابر متفاوت است.

| تابع | رفتار در مقدار مساوی | نمونه رتبه برای امتیازهای ۹۰، ۹۰، ۸۰ |
|---|---|---|
| `ROW_NUMBER()` | برای همه ردیف‌ها شماره متفاوت | ۱، ۲، ۳ |
| `RANK()` | امتیاز مساوی، رتبه مساوی؛ رتبه بعدی پرش دارد | ۱، ۱، ۳ |
| `DENSE_RANK()` | امتیاز مساوی، رتبه مساوی؛ بدون پرش | ۱، ۱، ۲ |

```
SELECT
    product_id,
    total_sales,
    ROW_NUMBER() OVER(
ORDER BY
    total_sales DESC,
    product_id) AS row_no, RANK() OVER(
ORDER BY
    total_sales DESC) AS sales_rank, DENSE_RANK() OVER(
```

```
ORDER BY
    total_sales DESC) AS dense_sales_rank
FROM
    product_sales;
```

**مثال اضافی:** جدول `product_sales` در این مثال یک خروجی تجمیع‌شده فرضی است و در فایل‌های جلسه به‌عنوان جدول قطعی معرفی نشده است. برای `RANK` و `DENSE_RANK` عمداً `product_id` به ترتیب امتیاز اضافه نشده تا امتیازهای برابر، رتبه برابر بگیرند.

## ۸.۲ حذف تکراری‌ها

یکی از الگوهای معرفی‌شده، شماره‌گذاری ردیف‌های تکراری بر اساس کلید منطقی و نگه‌داشتن ردیف مناسب بود. لازم است ابتدا تعریف کنیم تکراری یعنی «یک ایمیل»، «یک سفارش» یا «یک رویداد با شناسه منبع»؛ حذف بدون تعیین کلید ممکن است داده معتبر را از بین ببرد.

```
WITH ranked AS(
    SELECT
        customer_id,
        email,
        ROW_NUMBER() OVER(PARTITION BY email
    ORDER BY
        customer_id) AS rn
    FROM
        customers
)
SELECT
    customer_id,
    email
FROM
    ranked
WHERE
    rn = 1;
```

- **تکمیل فنی:** مثال بالا فقط رکوردهای انتخابی را نشان می‌دهد و چیزی حذف نمی‌کند. `NULL`ها و یکسان‌سازی حروف ایمیل باید بر اساس قرارداد داده بررسی شوند. برای حذف واقعی از پایگاه داده، ابتدا Preview, Backup و Transaction لازم است.

## ۸.۳ رشد ماهانه و جمع تجمعی

برای رشد نسبت به ماه گذشته، نخست مبلغ فروش هر ماه جداگانه محاسبه می‌شود، سپس با `LAG` مقدار ماه پیش در کنار ماه فعلی قرار می‌گیرد. `SUM OVER` نیز می‌تواند جمع تجمعی دوره‌ها را بسازد.

```
WITH monthly AS(
    SELECT
        DATE_TRUNC('month', order_at) ::date AS month_start,
        SUM(order_amount) AS monthly_sales
    FROM
        orders
    GROUP BY
        DATE_TRUNC('month', order_at) ::date
), trend AS(
    SELECT
        month_start,
        monthly_sales,
        LAG(monthly_sales) OVER(
    ORDER BY
        month_start) AS prev_sales, SUM(monthly_sales) OVER(
    ORDER BY
        month_start ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW) AS cumulative_sales
    FROM
        monthly
)
SELECT
    month_start,
    monthly_sales,
    prev_sales,
    cumulative_sales,
    ROUND(100.0 *(monthly_sales - prev_sales) / NULLIF(prev_s
ales, 0), 2) AS growth_pct
FROM
    trend
ORDER BY
    month_start;
```

- **تکمیل فنی:** اگر ماهی هیچ رکوردی نداشته باشد، `LAG` به ماهِ قبلی موجود در جدول اشاره می‌کند، نه الزاماً ماه تقویمی قبل. برای تحلیل ماهانه منظم، معمولاً باید جدول تقویم یا تولید همه ماه‌ها و سپس `LEFT JOIN` به نتیجه تجمیعی را در نظر گرفت.

### ۸.۴ قیف مشتری، Point-in-Time و الگوهای پیشرفته

مدرس مسئله قیف مشتریان یک ماه مشخص را مثال زد: ابتدا گروه مشتریانی را تعیین کنیم که در سپتامبر وارد شده‌اند و سپس بررسی کنیم چند نفر از آنان بعداً به مراحل خرید و پرداخت رسیده‌اند. تأکید کلاس بر **ثابت نگه‌داشتن جامعه اولیه** و تغییر ندادن ناخواسته آن در مرحله‌های بعد است.

در مسئله Point-in-Time نیز اگر برای یک پیش‌بینی تاریخی از داده‌ای استفاده کنیم که در آن زمان هنوز در دسترس نبوده، نتیجه آزمایش گمراه‌کننده می‌شود. مدرس از Sessionization و الگوی Gaps and Islands هم نام برد، ولی حل کاملشان را به تمرین‌های پیشرفته واگذار کرد.

- **تکمیل فنی:** جلوگیری از Data Leakage در تحلیل تاریخی فقط با فیلترکردن `event_time` ممکن نیست؛ زمان دسترس‌پذیری داده (`available_at`) و تغییرات بازنویسی‌شده گذشته نیز ممکن است مهم باشند. این تکمیل فراتر از شرح اجمالی کلاس است.

## ۹. پیوند SQL با مهندسی داده: Grain, Fact و Dimension

مدرس در پایان بحث را از Query به مدل‌سازی داده وصل کرد: پرسش «هر ردیف جدول نماینده چیست؟» باید پیش از JOIN, GROUP BY یا تولید گزارش پاسخ داده شود.

| مفهوم | معنی در مثال فروشگاهی | اثر بر Query |
|---|---|---|
| Grain | در `orders`، هر ردیف یک سفارش | جمع سفارش‌ها بدون تکثیر |
| Fact | رویداد خرید و مبلغ سفارش | اندازه‌گیری و تجمیع |
| Dimension | مشتری، محصول، زمان | توصیف و دسته‌بندی رخداد |
| Star Schema | Fact در مرکز و Dimensionها در اطراف | مدل تحلیلی قابل فهم |

اگر گزارش «مجموع مبلغ سفارش» می‌خواهد، مقدار را در سطح سفارش محاسبه کنید. اگر گزارش «فروش هر محصول» می‌خواهد، ممکن است Grain موردنیاز قلم سفارش باشد و لازم باشد جمع در آن سطح تعریف شود. دانستن اینکه `customer_id` در هر جدول یکتا است یا تکرار می‌شود، به همان اندازه مهم است که Syntax را درست بنویسیم.

- **تکمیل فنی:** Fact الزاماً فقط ستون‌های عددی ندارد و Dimension هم الزاماً ثابت نمی‌ماند. Fact معمولاً کلیدهای ارجاعی، اندازه‌ها و گاهی ویژگی‌های رویداد دارد؛ Dimension می‌تواند تاریخچه تغییرات داشته باشد. انتخاب Grain باید پیش از طراحی Measure انجام شود.

## ۱۰. چک‌لیست پیش از تأیید Query

۱. هر ردیف جدول من دقیقاً نماینده چیست و کلید یکتایش کدام است؟

۲. پس از JOIN، تعداد ردیف‌ها و `COUNT(DISTINCT key)` چه تغییری کرده است؟

۳. آیا `LEFT JOIN` با شرط روی جدول سمت راست در `WHERE` محدود شده است؟

۴. با مقدارهای `NULL` در فیلتر، `NOT IN` و شمارش چه برخوردی شده است؟

۵. بازه زمانی نیمه‌باز است و منطقه زمانی مورد نظر مشخص شده است؟

۶. `ORDER BY` برای Window Function هنگام تساوی ترتیب قطعی دارد؟

۷. میانگین، درصد و مخرج تقسیم واقعاً در Grain مورد انتظار تعریف شده‌اند؟

۸. در عملیات تغییر داده، دامنه ردیف‌ها از قبل شمرده و تراکنش کنترل شده است؟

۹. Query پارامتری است و دسترسی کاربر یا Agent حداقلِ لازم است؟

۱۰. عدد نهایی با یک کنترل مستقل و نمونه‌های مرزی سنجیده شده است؟

## ۱۱. تجربه مدرس و توصیه‌های پایانی

مدرس بارها اشاره کرد که تسلط بر Window Functions و الگوهای تحلیلی با مرور یک اسلاید به دست نمی‌آید. تمرین‌های `ROW_NUMBER`، `LAG`، درصد از کل، فیلتر روی نتیجه CTE و تشخیص Fan-out باید با تغییر صورت‌مسئله تکرار شوند. در پایان، موضوعات پیشرفته مانند قیف مشتری، Point-in-Time, Sessionization و Gaps and Islands برای مطالعه بعدی معرفی شدند و جلسه با تأکید بر نقش Grain, Fact و Dimension در پایپ‌لاین‌های قابل‌اعتماد پایان یافت.

## ۱۲. جمع‌بندی

SQL فقط نوشتن یک عبارت `SELECT` نیست؛ باید بتوانیم **معنای ردیف‌ها**، زمان محاسبه، نوع JOIN، رفتار NULL و پیامد هر تغییر را تحلیل کنیم. `Window Function` اجازه می‌دهد اطلاعات گروه را کنار تک‌تک ردیف‌ها نگه داریم؛ اما برای صحت خروجی، همیشه باید Grain و Cardinality را کنترل کنیم. مهم‌ترین عادت حرفه‌ای این جلسه، مقایسه عدد حاصل با یک معیار مستقل پیش از اعتماد کردن به گزارش است.

## ۱۳. سوال‌های مرور

۱. چرا اجرای یک Query مجموعه‌ای معمولاً از ارسال هزاران Query جدا مناسب‌تر است؟

۲. تفاوت اصلی `GROUP BY` و Window Function چیست؟

۳. `PARTITION BY customer_id` چه کاری انجام می‌دهد؟

۴. چرا در مرتب‌سازی سفارش‌ها، افزودن `order_id` به `ORDER BY order_at` سودمند است؟

۵. برای نمایش مبلغ سفارش قبلی چه تابعی مناسب است؟

۶. چرا `ROW_NUMBER()` را معمولاً در `WHERE` همان Query فیلتر نمی‌کنیم؟

۷. Fan-out چگونه ممکن است جمع مبلغ سفارش را خراب کند؟

۸. چرا `COUNT(*)` پس از `LEFT JOIN` برای شمردن سفارش مناسب نیست؟

۹. شرط `WHERE o.status = 'paid'` پس از `LEFT JOIN` چه اثری دارد؟

۱۰. چرا `country <> 'Iran'` ردیف‌های NULL را برنمی‌گرداند؟

۱۱. چگونه NULL در زیرپرس‌وجوی `NOT IN` نتیجه را غیرمنتظره می‌کند؟

۱۲. مزیت شرط `< '2026-10-01'` نسبت به `BETWEEN` برای Timestamp چیست؟

۱۳. برای `UPDATE` حساس چه بررسی‌هایی قبل از `COMMIT` انجام می‌دهیم؟

۱۴. تفاوت `RANK` و `DENSE_RANK` در مقادیر مساوی چیست؟

۱۵. Grain چه نقشی در طراحی Fact و محاسبه Measure دارد؟

## ۱۴. پاسخ‌نامه کوتاه

۱. تعداد رفت‌وبرگشت کمتر و امکان بهینه‌سازی یکپارچه‌تر، البته بسته به طرح اجرا.

۲. `GROUP BY` گروه را به ردیف تجمیعی تبدیل می‌کند؛ Window Function معمولاً ردیف‌ها را نگه می‌دارد.

۳. پنجره را برای هر مشتری مستقل می‌کند.

۴. هنگام یکسان‌بودن زمان، ترتیب قطعی‌تری می‌دهد.

۵. با تابع `LAG(order_amount) OVER (...)` می‌توان مبلغ ردیف قبلی را خواند.

۶. Window Function بعد از فیلتر `WHERE` محاسبه می‌شود؛ از CTE استفاده می‌کنیم.

۷. JOIN یک‌به‌چند، ردیف سفارش را چند بار تکرار می‌کند.

۸. `COUNT(*)` ردیف NULL ساخته‌شده توسط JOIN را نیز حساب می‌کند؛ `COUNT(o.order_id)` مناسب‌تر است.

۹. مشتریان بدون سفارش پرداخت‌شده را حذف می‌کند؛ در صورت نیاز شرط باید داخل `ON` باشد.

۱۰. مقایسه با NULL نتیجه UNKNOWN می‌دهد.

۱۱. چون مقایسه با عضو NULL می‌تواند UNKNOWN شود، حتی پاسخ صفر ایجاد شود.

۱۲. تمام لحظه‌های روز آخر ماه را بدون وابستگی به دقت اعشار پوشش می‌دهد.

۱۳. شمارش کاندیدها، بررسی شرط و ردیف‌های برگشتی، تراکنش و تصمیم صریح درباره COMMIT/ROLLBACK.

۱۴. `RANK` بعد از تساوی پرش دارد؛ `DENSE_RANK` ندارد.

۱۵. تعیین می‌کند هر ردیف چه رویدادی است و کدام جمع یا JOIN معتبر خواهد بود.

## ۱۵. واژه‌نامه

| ردیف | اصطلاح | معنی کاربردی |
|---|---|---|
| ۱ | Set-based Thinking | حل مسئله روی مجموعه ردیف‌ها به‌جای حلقه‌های مستقل |
| ۲ | N+1 Query | تعداد زیاد Query کوچک به‌ازای هر موجودیت |
| ۳ | Window Function | تابعی که با حفظ ردیف، محاسبه پنجره‌ای انجام می‌دهد |
| ۴ | `OVER` | تعیین محدوده محاسبه تابع پنجره‌ای |
| ۵ | `PARTITION BY` | جداکردن پنجره‌ها بر اساس کلید |
| ۶ | `ORDER BY` | مشخص‌کردن ترتیب خروجی یا ترتیب در پنجره |
| ۷ | `ROW_NUMBER` | شماره ترتیبی یکتای هر ردیف در پنجره |
| ۸ | `LAG` | خواندن مقدار ردیف پیشین |
| ۹ | `LEAD` | خواندن مقدار ردیف بعدی |
| ۱۰ | Running Total | جمع تجمعی تا هر ردیف |
| ۱۱ | Window Frame | محدوده ردیف‌های مشارکت‌کننده در محاسبه |
| ۱۲ | CTE | عبارت نام‌دار نتیجه میانی با `WITH` |

| ردیف | اصطلاح | معنی کاربردی |
|---|---|---|
| ۱۳ | Fan-out | زیادشدن ناخواسته ردیف‌های حاصل از JOIN |
| ۱۴ | Cardinality | تعداد رابطه‌ها و میزان تطابق بین کلیدها |
| ۱۵ | Grain | معنای دقیق هر ردیف جدول |
| ۱۶ | `LEFT JOIN` | حفظ تمام ردیف‌های سمت چپ همراه تطابق‌های سمت راست |
| ۱۷ | `COUNT(DISTINCT)` | شمارش مقدارهای متمایز غیر NULL |
| ۱۸ | Three-valued Logic | منطق TRUE, FALSE و UNKNOWN |
| ۱۹ | `NOT EXISTS` | نبود ردیف مطابق در زیرپرس‌وجو |
| ۲۰ | Half-open Interval | بازه شامل ابتدا و خارج از انتها |
| ۲۱ | Transaction | مجموعه عملیات کنترل‌شده با COMMIT یا ROLLBACK |
| ۲۲ | `NULLIF` | برگرداندن NULL در صورت برابری دو عبارت |
| ۲۳ | Parameterized Query | Query با پارامتر امنِ متصل‌شده به Driver |
| ۲۴ | `RANK` | رتبه با پرش پس از تساوی |
| ۲۵ | `DENSE_RANK` | رتبه بدون پرش پس از تساوی |
| ۲۶ | Data Leakage | استفاده از اطلاعات ناموجود در زمان تصمیم تاریخی |
| ۲۷ | Fact Table | جدول رخدادها و Measureها در Grain مشخص |
| ۲۸ | Dimension Table | جدول توصیف‌کننده موجودیت‌ها و دسته‌بندی‌ها |
| ۲۹ | Star Schema | مدل تحلیلی Fact مرکزی و Dimensionهای پیرامونی |
| ۳۰ | Gaps and Islands | الگوی یافتن بازه‌های پیوسته و فاصله‌ها |